

Pandas — Fiche de révision

Concours ANP — Cadre Data Scientist

1 Lecture et exploration d'un DataFrame

1.1 read_csv()

Charge un fichier CSV dans un **DataFrame** (structure tabulaire, lignes + colonnes).

```
import pandas as pd

df = pd.read_csv("navires.csv")
df = pd.read_csv("navires.csv", sep=";", encoding="utf-8")
```

1.2 head()

Affiche les premières lignes (5 par défaut) — première chose à faire pour inspecter un jeu de données.

```
df.head() # 5 premières lignes
df.head(10) # 10 premières lignes
df.tail(3) # 3 dernières lignes
```

1.3 info()

Résume la structure du DataFrame : nombre de lignes, colonnes, types (**dtype**), valeurs non nulles, mémoire utilisée.

```
df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 1000 entries, 0 to 999
# Data columns (total 4 columns):
# # Column Non-Null Count Dtype
# 0 nom 1000 non-null object
# 1 tirant_eau 950 non-null float64
```

— Premier réflexe pour détecter des **valeurs manquantes** (colonnes avec moins de non-null que le total de lignes).

1.4 describe()

Statistiques descriptives des colonnes numériques : count, mean, std, min, quartiles, max.

```
df.describe()
df.describe(include="all") # inclut aussi les colonnes non numériques
```

— Permet de repérer rapidement des **outliers** (min/max éloignés des quartiles) ou une distribution anormale.

2 Sélection : loc et iloc

2.1 loc — sélection par label

```
df.loc[0] # ligne d'index 0
df.loc[0, "nom"] # valeur a la ligne 0, colonne "nom"
df.loc[0:5, ["nom", "tirant_eau"]] # lignes 0 a 5 (inclus), colonnes choisies
df.loc[df["tirant_eau"] > 14] # filtre conditionnel (booleen)
```

2.2 iloc — sélection par position entière

```
df.iloc[0] # premiere ligne (position 0)
df.iloc[0, 1] # ligne 0, colonne d'index 1
df.iloc[0:5, 0:2] # lignes 0 a 4, colonnes 0 a 1 (borne haute exclue)
```

- **loc** : sélection par **nom** (label) de ligne/colonne, borne haute **incluse**.
- **iloc** : sélection par **position** entière (comme une list Python), borne haute **exclue**.
- Erreur fréquente : confondre les deux quand l'index du DataFrame n'est pas une simple séquence 0,1,2,...

3 Agrégation : groupby()

Regroupe les lignes selon une ou plusieurs colonnes, puis applique une fonction d'agrégation (paradigme *split-apply-combine*).

```
df.groupby("port")["tirant_eau"].mean() # moyenne par port
df.groupby("port").agg({"tirant_eau": "mean", "conteneurs": "sum"})
df.groupby(["port", "type_navire"]).size() # nombre de lignes par groupe
```

- Trois étapes : **split** (découper en groupes), **apply** (appliquer une fonction à chaque groupe), **combine** (recombinaison des résultats).
- Équivalent du GROUP BY en SQL.

4 Jointure : merge()

Combine deux DataFrames sur une ou plusieurs colonnes communes (équivalent du JOIN en SQL).

```
pd.merge(navires, ports, on="port_id", how="inner")
pd.merge(navires, ports, on="port_id", how="left") # garde toutes les lignes de gauche
pd.merge(navires, ports, left_on="port_id", right_on="id", how="outer")
```

- **how** : "inner" (intersection), "left", "right", "outer" (union) — même logique que les jointures SQL.
- **on** si le nom de colonne est identique dans les deux DataFrames, sinon **left_on/right_on**.

5 Nettoyage des données

5.1 fillna()

Remplace les valeurs manquantes (NaN) par une valeur donnée.

```
df["tirant_eau"].fillna(0, inplace=True)
df["tirant_eau"].fillna(df["tirant_eau"].mean(), inplace=True) # imputation par la moyenne
df.fillna(method="ffill") # propage la derniere valeur valide vers l'avant
```

5.2 dropna()

Supprime les lignes (ou colonnes) contenant des valeurs manquantes.

```
df.dropna() # supprime toute ligne avec au moins un NaN
df.dropna(subset=["tirant_eau"]) # seulement si NaN dans cette colonne
df.dropna(axis=1) # supprime les colonnes avec des NaN
```

- Choix `fillna` vs `dropna` : `dropna` perd de l'information (lignes entières supprimées) ; `fillna` garde les lignes mais introduit une estimation — le choix dépend du volume de données manquantes et de l'impact sur l'analyse.

5.3 drop_duplicates()

Supprime les lignes dupliquées.

```
df.drop_duplicates() # doublons sur toutes les colonnes
df.drop_duplicates(subset=["nom"]) # doublons bases sur une colonne
df.drop_duplicates(keep="last") # garde la dernière occurrence
```

6 Tri : sort_values()

Trie le DataFrame selon une ou plusieurs colonnes.

```
df.sort_values("tirant_eau") # ordre croissant
df.sort_values("tirant_eau", ascending=False) # ordre décroissant
df.sort_values(["port", "tirant_eau"], ascending=[True, False]) # tri multi-colonnes
```

7 Questions types (révision active)

1. Que fait `df.info()` et pourquoi le lancer avant toute analyse ?
2. Quelle est la différence entre `loc` et `iloc` ?
3. Que signifie *split-apply-combine* pour `groupby()` ?
4. Quelle est la différence entre les types de jointure `"inner"`, `"left"` et `"outer"` dans `merge()` ?
5. Quand préférer `fillna()` à `dropna()` ?
6. Comment supprimer les doublons d'un DataFrame en se basant sur une seule colonne ?

8 Corrigé des questions types

Q1. Que fait `df.info()` et pourquoi le lancer avant toute analyse ?

`df.info()` affiche le nombre de lignes, la liste des colonnes, leur type (`dtype`) et le nombre de valeurs non nulles par colonne. C'est le premier réflexe en exploration de données car il révèle immédiatement les **valeurs manquantes** (colonne avec moins de non-null que le total de lignes) et les **types incohérents** (ex. une colonne numérique lue comme texte), avant même de calculer la moindre statistique.

Q2. Différence entre `loc` et `iloc` ?

`loc` sélectionne par **label** (nom de ligne/colonne), avec une borne haute **incluse** dans un slice. `iloc` sélectionne par **position entière** (comme l'indexation d'une list Python), avec une borne haute **exclue**. La confusion apparaît surtout quand l'index du DataFrame n'est pas une séquence simple 0,1,2,... (par exemple après un filtre ou un tri qui ne réinitialise pas l'index).

Q3. Que signifie *split-apply-combine* pour `groupby()` ?

C'est le paradigme sous-jacent à `groupby()` : **split** découpe le DataFrame en sous-groupes selon les valeurs d'une ou plusieurs colonnes (ex. par port) ; **apply** applique une fonction d'agrégation à chaque groupe indépendamment (moyenne, somme, compte...) ; **combine** recombine les résultats de chaque groupe en une seule structure de sortie. C'est l'équivalent pandas du `GROUP BY SQL`.

Q4. Différence inner / left / outer dans merge() ?

"inner" ne garde que les lignes dont la clé de jointure existe dans les **deux** DataFrames (intersection). "left" garde **toutes** les lignes du DataFrame de gauche, avec des **NaN** pour les colonnes de droite si aucune correspondance n'existe. "outer" garde toutes les lignes des deux côtés (union), avec des **NaN** là où une correspondance manque. Le choix dépend de si on veut préserver les lignes sans correspondance ou seulement les lignes communes.

Q5. Quand préférer fillna() à dropna() ?

dropna() supprime des lignes entières, donc de l'information potentiellement utile sur d'autres colonnes — à éviter si beaucoup de lignes ont au moins une valeur manquante, ou si chaque ligne est précieuse (petit jeu de données). **fillna()** conserve toutes les lignes en estimant la valeur manquante (moyenne, médiane, valeur précédente...), au prix d'introduire une approximation. On privilégie **fillna** quand le volume de données manquantes est important ou que perdre des lignes fausserait l'analyse, et **dropna** quand les valeurs manquantes sont rares et qu'on préfère des données garanties exactes.

Q6. Supprimer les doublons sur une seule colonne ?

```
df.drop_duplicates(subset=["nom"])
```

Le paramètre **subset** restreint la détection de doublon à la ou les colonnes indiquées, au lieu de comparer toutes les colonnes du DataFrame. Par défaut, **keep="first"** conserve la première occurrence rencontrée ; **keep="last"** permet de garder la dernière à la place.